Design Pattern Analysis in a Node.js/Express Ride-Sharing Application

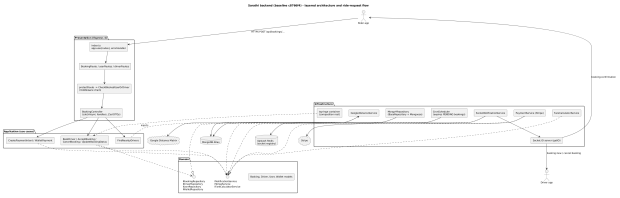
23CSE455 Case Study – Team A6

Dilip Kumar Reddy Sai Sameer M Kumar K Harish S Leela Shankar

2026-10-11

Application: Sarathi backend

- github.com/AkhilMM2000/sarathi-backend, commit `c8766f4`
- TypeScript, Express 4, Mongoose, tsyringe DI, Socket.IO + Redis, Stripe, Google Distance Matrix
- Areas studied: rider, driver, trip, payment, matching, location, notification
- Matching: 30 km `$geoNear` → 20 km road distance → broadcast; first driver to accept wins
- No upstream license; no tests – we wrote 14 characterization tests



Part 1 – patterns actually used (baseline)

Pattern	Inst.	Example instance
Layered Architecture	1	Ride Booking Request Flow
Dependency Injection	1	tsyringe composition root
Singleton	1	Shared Socket.IO server (`getIO`)
Repository	5	Trip, Driver (geo query), Rider, Wallet, Review
Data Mapper	1	Driver review mapping
Adapter	3	Socket.IO notification, Stripe payment, Stripe Connect
Gateway	1	Google Distance Matrix
Observer	1	Ride event notification (Redis socket registry)
Chain of Responsibility	1	`protectRoute` → `checkBlocked` → controller
Decorator	1	`catchAsync` error forwarding

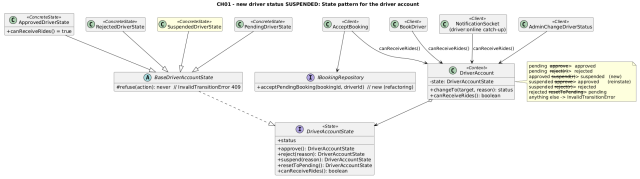
10 types, 16 instances. Rejected: Strategy, State, Command, Template Method, Factory Method, Facade. Uncertain: unused Google Maps and Twilio adapters.

What was missing – and the defects it caused

- Fare by ride type: `if/else` in `FareCalculatorService`; Rs 10/km duplicated in `UpdateBookingStatus`
- Trip and driver status: plain strings + scattered checks
  - a **COMPLETED trip could be cancelled** (baseline test passes)
  - an admin could move a **rejected driver straight to approved**
- `AcceptBooking` queried Mongoose directly (layer violation)

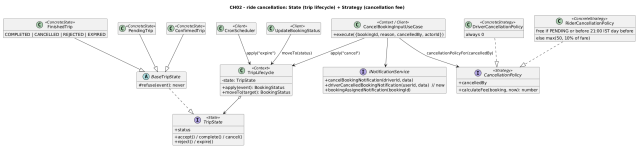
CH01 – new driver status SUSPENDED (State)

- `DriverAccountState`: Pending, Approved, Rejected, Suspended; context `DriverAccount`
- Illegal transitions → 409, nothing saved or sent
- Suspended drivers: not matched, cannot accept, no catch-up requests, no login
- Refactoring: Move Method → `acceptPendingBooking`
- Tests 22/22



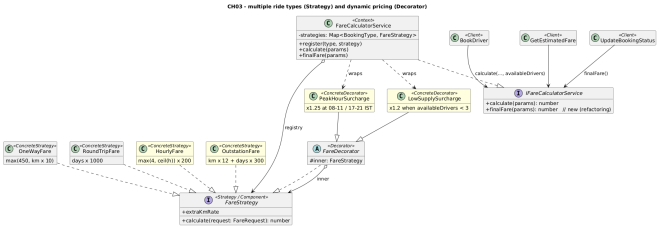
CH02 – ride cancellation (State + Strategy)

- `TripLifecycle`: Pending, Confirmed, Finished – used by cancel, status update, expiry
- `CancellationPolicy`: rider late fee (10%, min Rs 50 after 21:00 IST day before); driver free
- Rider *and* driver can cancel only their own rides
- New event `booking:cancelledByDriver`
- Tests 33/33



CH03 – ride types + dynamic pricing (Strategy + Decorator)

- New HOURLY and OUTSTATION fares; one class per ride type
- `PeakHourSurcharge` $\times 1.25$, `LowSupplySurcharge` $\times 1.2$ wrap any strategy
- `FareCalculatorService` CC 6 $\rightarrow$ 2
- 6 baseline fare tests pass unchanged
- Tests 41/41



How the pattern set changed

Snapshot	Types	Instances	Change
Original	10	16	–
After CH01	11	17	+ State (driver account)
After CH02	12	19	+ State (trip), + Strategy (cancellation fee)
After CH03	12	21	+ Strategy (ride fare), + Decorator (dynamic pricing)

Extended: notification Adapter + Observer, middleware chain, trip repository, DI. Nothing removed. Cost: +448 NLOC overall; some use cases more complex (ownership, supply rules). LOC alone is not the verdict – the defects are now tested.

Part 3 – same problem in four languages

	Java 25	Python 3.13	Node 22	C++20 (g++ 13)
Strategy LOC / CC / ms	57 / 3 / 103	51 / 3 / 60	47 / 3 / 49	69 / 3 / 2.1
State LOC / CC / ms	58 / 6 / 98	52 / 2 / 31	41 / 2 / 45	57 / 6 / 1.9
Observer LOC / CC / ms	33 / 3 / 111	31 / 3 / 64	24 / 2 / 46	40 / 3 / 2.3

All 12 pass: Strategy 8/8, State 8/8, Observer 6/6. Time = median of 5 full test processes (start-up dominated).

Part 3 – what differed

- **Roles:** nominal interfaces checked by the compiler (Java, C++) vs. `Protocol`/duck typing (Python, JS)
- **Dispatch:** `getattr`/`state[event]()` removes the event switch (CC 2 vs 6)
- **Ownership:** only C++ must say who owns strategies/states (`unique_ptr`) and that the hub must not keep observers alive (`weak_ptr`)
- **Start-up:** JVM ≈100 ms, Node ≈47 ms, CPython 31–64 ms (imports matter), native ≈2 ms

LLM use and limitations

- LLM records kept only for real runs: prompt, input, raw output, verification (`prompts/`, `data/llm.csv`)
- First Claude pass: partially correct – missed Gateway and `catchAsync` Decorator, over-counted unused adapters
- Every pattern accepted only from traced code + tests, never from a name or an LLM claim
- Limitations: no live run against MongoDB/Redis/Stripe/Google; dependency drift (7 pre-existing `tsc` errors); no upstream license